

0 Quick Index / 快速索引

**Ch1** (总览/中断/缓存) : CPU registers, interrupts, memory hierarchy, cache, DMA.

**Ch2** (OS 目标/接口/模式) : convenience, efficiency, evolution; ISA/ABI/API; system call; user/kernel mode.

**Ch3** (进程/状态/切换) : process image, PCB, five/seven-state model, suspend, trap, process switch.

**Ch4** (线程模型) : thread vs process, ULT/KLT, jacketing, shared resources.

**Ch5** (并发同步) : race condition, critical section, mutex, semaphore, producer/consumer.

**Ch6** (死锁处理) : four conditions, prevention, avoidance, detection, banker, safe state.

**Ch7** (内存管理) : relocation, partitioning, fragmentation, buddy, paging, address translation.

**Ch8** (虚拟内存) : page fault, resident set, TLB, OPT/LRU/FIFO/Clock, working set, thrashing.

**Ch9** (单处理器调度) : long/medium/short scheduler, FCFS, RR, SPN/SJF, SRT, HRRN, feedback.

**Ch10** (多核/实时) : load sharing, gang scheduling, dedicated/dynamic scheduling, hard/soft real-time.

**Ch12** (文件管理) : file organization, directory, B-tree, blocking, contiguous/chained/indexed allocation.

考试查找路线: 先按章定位, 再扫绿色关键词后的中文括注; 计算题优先看 Ch7 地址转换、Ch8 置换/TLB、Ch9 调度甘特图、Ch12 B-tree/分配。

1 Computer System Overview

Main Elements and Registers

**Processor** (处理器/CPU) : executes instructions and controls system operation.  
**Main memory** (主存/当前程序数据) : stores currently used programs and data.

**I/O modules** (输入输出模块) : move data between the computer and external devices.

**System bus** (数据/地址/控制线) : carries data, address, and control signals among modules.

**User-visible registers** (用户可见寄存器) : referenced by machine instructions to reduce memory access.

**Control/status registers** (控制/状态寄存器) : PC, IR, MAR, MBR, PSW; used by CPU and OS to control execution.

Instruction Actions

- Processor-memory** (访存指令) : load/store between CPU and memory.

- Processor-I/O** (I/O 传送) : transfer data between CPU and I/O modules.
- Data processing** (算术/逻辑) : arithmetic or logical operations.
- Control** (改变执行流) : branch, jump, call, return; changes execution order.

Interrupts

Purpose and Types

**Interrupt** (中断/提高 CPU 利用率) : a mechanism by which I/O, timer, program error, or hardware failure interrupts normal processor execution. Main purpose: improve processor utilization.

**Program** (程序异常) : divide by zero, overflow, illegal instruction.  
**Timer** (时钟中断) : lets the OS regain control periodically.  
**I/O** (设备完成/就绪) : device reports completion or readiness.  
**Hardware failure** (硬件故障) : power failure, memory parity error.

Processing Sequence

- Device issues interrupt; CPU finishes current instruction.
- CPU acknowledges, saves PC and PSW, and loads ISR address.
- ISR saves remaining state, handles the event, restores state.
- Old PC and PSW are restored; interrupted program resumes.

PC tells where to resume; PSW records processor state. Polling repeatedly tests device flags; interrupts let the device notify the CPU.

Multiple Interrupts

**Sequential** (顺序处理) : disable interrupts during ISR; new interrupts stay pending. Simple but may delay urgent events.

**Nested** (嵌套/优先级) : assign priorities; a higher-priority interrupt may interrupt a lower-priority ISR.

Memory, Cache, I/O

Memory Hierarchy

Higher levels are **faster** (更快), **smaller** (更小), and **more expensive per bit** (每位更贵); lower levels are larger and slower. Distinguish levels by capacity, access time, cost per bit, and frequency of access.

$$T_{avg} = \sum_i \left( \prod_{j < i} (1 - H_j) \right) T_i$$

$$C_{avg} = \sum C_i S_i / \sum S_i$$

Cache and Locality

**Cache** (高速缓存) : small fast memory between processor and main memory; stores recently or frequently used blocks to reduce average access time.

**Temporal locality** (时间局部性/最近还会用) : a referenced item is likely to be referenced again soon; keep recent items in fast memory.

**Spatial locality** (空间局部性/附近也会用) : nearby addresses are likely to be referenced soon; fetch a block, not only one word.

I/O and DMA

**Programmed I/O** (程序控制 I/O/轮询) : CPU waits by polling flags such as FGI/FGO.

**Interrupt-driven I/O** (中断驱动 I/O) : IEN enables the device to interrupt when ready.

**DMA** (直接存储器访问) : transfers data directly between I/O device and memory; DMA gets high memory priority because device delay may cause data loss, while CPU delay usually only slows execution.

Chapter 1 Exam Questions

Review 1.1–1.10

**Q:** List the four main computer elements.  
**A:** Processor, main memory, I/O modules, system bus.

**Q:** Define the two register categories.  
**A:** User-visible registers; control and status registers.

**Q:** What is an interrupt?  
**A:** A mechanism that lets another module interrupt normal CPU execution so an ISR can handle the event.

**Q:** How are multiple interrupts handled?  
**A:** Sequential processing disables interrupts during ISR; nested processing uses priorities.

**Q:** What distinguishes memory hierarchy levels?  
**A:** Capacity, access time, cost per bit, and access frequency.

**Q:** Multiprocessor vs multicore?  
**A:** Multiprocessor has multiple processors; multicore has multiple cores on one chip.

Problem Stems and Answers

**1.1 I/O program:** Load dev5, add M[940], store dev6: dev6 receives 0005.

**1.2 MAR/MBR:** Expand Fig. 1.4 execution: fetch via MAR/MBR; final M[941] = 0005.

**1.3 32-bit instr.:** 24-bit address field gives

16 MB; PC = 24 bits, IR = 32 bits.

**1.4 16-bit addr.:** 16-bit memory: 128 KiB; 8-bit memory: 64 KiB; 8-bit port number: 256 ports.

**1.5 bus rate:** 8 MHz, 16-bit bus, 4 clocks/bus cycle gives 4 MB/s; wider bus or doubled clock gives 8 MB/s.

**1.6 Teletype I/O:** FGI/FGO polling wastes CPU; IEN enables interrupt-driven I/O.

**1.7 DMA priority:** DMA gets memory priority because device delay may lose data; CPU delay mostly slows execution.

**1.8 DMA slowdown:** 9600 bps DMA slows 1 MIPS CPU by about 0.12%.

**1.9 I/O rate:** Programmed I/O = 25,000 words/s; DMA =  $2.15 \times 10^6$  words/s.

**1.10 locality code:** Spatial: sequential a[i]; temporal: repeated same a[i] in inner loop.

**1.11 n-level memory:** Use  $T_{avg} = \sum_i (\prod_{j < i} (1 - H_j)) T_i$ ,  $C_{avg} = \sum C_i S_i / \sum S_i$ .

**1.12 cost/hit:** Main memory  $\approx$  \$83.89; cache-tech  $\approx$  \$838.86;  $H \approx$  99.17%.

**1.13 cache/disk:** Average access time  $\approx$  480,026 ns.

**1.14 stack PC:** No; the PC cannot generally be eliminated by using the stack top.

2.1 OS Objectives and Functions

Definition

**Operating system** (操作系统) : a program that controls application execution and acts as an interface between applications and computer hardware.

Three Objectives

**Convenience** (方便使用) : makes the computer easier to use.

**Efficiency** (资源效率) : uses CPU, memory, I/O, files, storage, and network resources efficiently.

**Evolution** (可演化/可维护) : permits new functions, tests, fixes, and hardware support without disrupting service.

Two Main Roles

- User/computer interface** (用户接口) : hides hardware details and provides usable services.
- Resource manager** (资源管理器) : allocates processor, memory, I/O devices, files, storage, and network resources.

OS Services

|                                     |                                                               |
|-------------------------------------|---------------------------------------------------------------|
| <b>Program development</b> (程序开发服务) | editors, debuggers, compilers, libraries.                     |
| <b>Program execution</b> (程序执行服务)   | loads code/data, initializes files, I/O, and resources.       |
| <b>I/O access</b> (I/O 统一访问)        | uniform device interface, e.g. read(), write().               |
| <b>File access</b> (文件访问)           | file organization, permissions, protection.                   |
| <b>System access</b> (系统访问/权限)      | login, authorization, resource contention control.            |
| <b>Error handling</b> (错误处理)        | detect hardware/software errors; terminate, retry, or report. |
| <b>Accounting</b> (记账/性能统计)         | collect usage/performance data; support billing and tuning.   |

Interfaces: ISA vs ABI vs API

**ISA** (指令集接口) (指 CPU 指令集接口) : hardware/software boundary; set user ISA vs system ISA.

**ABI** (二进制接口) (binary compatibility) : system-call interface, registers, calling convention, binary format.

**API** (源代码接口) (源代码接口) : source-level calls library functions used by programmers, e.g. open(), malloc().

Memory hook: API = coding; ABI = compiled binary execution; ISA = CPU instructions.

Kernel, Control, Evolution

Kernel / Nucleus

**Kernel** (内核/常驻核心) : central OS part, normally resident in main memory, containing the most frequently used OS functions: process, memory, I/O, file, interrupt, and system-call handling.

How the OS Controls the CPU

The OS is also software. It gains control, schedules and allocates resources, gives control to user programs, then regains control through interrupts, system calls, or exceptions.

Why OSs Must Evolve

- Hardware upgrades** (硬件升级) : new processors/devices require new support.
- New services** (新增服务) : users and

- administrators need new tools and features.
- **Fixes** (修复错误) : bugs must be repaired without destabilizing old services.

### System Calls and Modes

#### System Calls

**System call** (系统调用/进内核) : controlled entry point for a user program to request OS services such as file I/O, process creation, memory allocation, communication, and protection.

#### Dual-Mode Operation

**User mode** (用户态/低权限) low privilege; applications cannot directly execute privileged instructions.  
**Kernel mode** (内核态/高权限) high privilege; OS kernel performs protected operations.

Flow: a user program requests a system call/trap; the kernel performs the service and returns to user mode.

### PDF Stems and Answers

- R2.1:** Stem: three OS-design objectives. A: convenience, efficiency, evolution.  
**R2.2:** Stem: kernel of an OS. A: memory-resident core with frequent OS functions.  
**P2.4:** Stem: system-call purpose and dual-mode relation. A: safe service request; user mode traps to kernel mode, then returns.  
**P2.5:** Stem: OS/390 SRM checks untouched page frames. A: find inactive pages, guide replacement/reallocation, detect memory pressure or thrashing.  
**R2.6:** Stem: five storage duties. A: isolation, allocation, modular support, protection/access control, long-term storage.

### Chapter 2 Fast Recall

- OS = application controller + hardware interface + resource manager.
- OS hides hardware details behind services and interfaces.
- Important services: development, execution, I/O, files, access control, errors, accounting.
- Kernel holds common privileged OS functions in memory.
- System calls are the legal bridge from user mode to kernel mode.
- Evolution needs modular design, clear interfaces, and maintainability.

### 3 Process Description and Control

#### Process, Image, PCB

**Process** (进程/执行中的程序) : a program in execution; an entity that can be assigned to and executed by a processor.

**Process image** (进程映像) : program code, data, stack, and process attributes.

**PCB**(进程控制块): OS record holding enough information to stop and later resume a process as if it had not been interrupted.

**ID** (标识信息) PID, parent PID, user ID.  
**CPU state** (处理器状态) registers, PC, PSW, stack pointer.  
**Control** (控制信息) state, priority, scheduling links, memory pointers, I/O status, accounting.

**Instruction trace** (指令轨迹) : sequence of instructions actually executed by a process.

**OS tables** (OS 管理表) : memory, I/O, file, and process tables.

#### Five-State Model

**New** (新建) created but not yet admitted.  
**Ready** (就绪/只差 CPU) can run once assigned a CPU; lacks only CPU.  
**Running** (运行) currently executing on the CPU.  
**Blocked** (阻塞/等事件) waits for an event such as I/O completion.  
**Exit** (退出) terminated or released.

**Admit** (接纳) : New → Ready. **Dispatch** (派遣) : Ready → Running. **Timeout/preempt** (超时/抢占) : Running → Ready. **Event wait** (等待事件) : Running → Blocked. **Event occurs** (事件发生) : Blocked → Ready. **Release** (释放) : Running → Exit.  
Ready = waiting for CPU; Blocked = waiting for an event. An event-completed blocked process returns to Ready, not directly to Running.

#### Suspension and Swapping

**Swapping** (交换/内存磁盘搬移) : moving a process between main memory and disk to free memory and improve CPU use.

**Suspended** (挂起/不可立即执行) : not immediately available for execution; may or may not be waiting for an event.

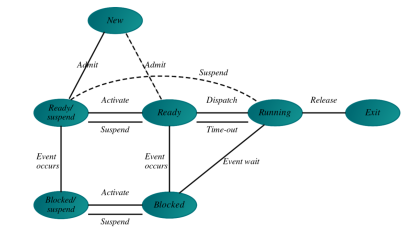
**Ready/Suspend** (就绪挂起) swapped out; event condition satisfied; can run after swap-in.  
**Blocked/-Suspend** (阻塞挂起) swapped out and still waiting for an event.

**Reasons** (挂起原因) swapping, OS reason, interactive user request, timing, parent request.

**Traits** (挂起特征) not executable now; may wait on event; agent-suspended; needs explicit activation.

**Queues** (队列组织) : use one Ready queue plus per-event Blocked queues; suspended states use Ready/Suspend and per-event Blocked/Suspend queues.

#### Seven-State Diagram



#### Control, Modes, Switching

**Creation steps** (创建进程步骤) : assign PID; allocate process space; initialize PCB; link into ready or suspend queues; create/expand accounting and other tables.

**User mode** (用户态) : low privilege; cannot execute privileged instructions. **Kernel/System mode** (内核/系统态) : high privilege for protected OS operations.

**Interrupt** (外部异步) : asynchronous external event. **Trap** (当前指令异常) : exception caused by the current instruction. **Supervisor call** (访管调用) : explicit request for an OS service.

**Mode switch** (模式切换) : changes privilege mode, possibly same process. **Process switch** (进程切换) : CPU changes from one process to another.

**Process switch work** (上下文切换工作) : save old PC, PSW, registers into PCB; update state and queues; choose next process; load next PCB/context.

### Chapter 3 Review Q&A

- R3.1 Trace:** Stem: instruction trace. A: executed instruction sequence of a process.  
**R3.2 Creation:** Stem: process creation events. A: new batch job, interactive logon,

OS-created service, spawned child.

**R3.3 States:** Stem: Fig. 3.6 states. A: New, Ready, Running, Blocked, Exit.

**R3.4 Preempt:** Stem: preempt a process. A: reclaim CPU before the process voluntarily yields or finishes.

**R3.5 Swapping:** Stem: swapping and purpose. A: move process memory to/from disk to free memory and improve CPU use.

**R3.6 Two blocked:** Stem: why Fig. 3.9b has two blocked states. A: resident Blocked vs swapped-out Blocked/Suspend.

**R3.7 Suspended:** Stem: four suspended-process traits. A: not executable now; may wait; agent-suspended; explicitly activated.

**R3.8 OS tables:** Stem: OS management tables. A: memory, I/O, file, process.

**R3.9 PCB info:** Stem: three PCB categories. A: identification, processor state, process control.

**R3.10 Modes:** Stem: why user/kernel modes. A: protect OS, PCBs, memory, I/O, privileged instructions.

**R3.11 Create:** Stem: OS steps to create process. A: PID, space, PCB, links, other tables.

**R3.12 Int/trap:** Stem: interrupt vs trap. A: external asynchronous event vs instruction-caused exception.

**R3.13 Examples:** Stem: three interrupts. A: clock, I/O, memory fault.

**R3.14 Switches:** Stem: mode switch vs process switch. A: privilege change vs changing the running process.

### Chapter 3 Problem Q&A

**P3.1 Transitions:** Stem: R/RUN/B/NR transition table. A: 1 dispatch; 2 preempt; 3 event wait; 4 event occurs; 5 ready swap-out; 6 blocked swap-out.

**P3.2 Timeline:** Stem: process states at  $t = 22, 37, 47$ . A:  $t = 22$ : P1 B(d3 read), P3 B(d2 read), P7 B(d3 write), P5 likely running.  $t = 37$ : P1/P3 ready, P5 Blocked/Suspend, P7 blocked, P8 running.  $t = 47$ : P5 ready, P7 blocked, P8 exit; one ready process may run.

**P3.3 Seven states:** Stem: possible/impossible Fig. 3.9b transitions. A: Possible direct moves: N→R/RS/E; R→Run/RS/E; Run→R/B/RS/E; B→R/BS/E; RS→R/E; BS→B/RS/E; all other direct moves are impossible.

**P3.4 Queues:** Stem: seven-state queueing diagram. A: Ready queue, processor, per-event Blocked queues, Ready/Suspend queue, and per-event Blocked/Suspend queues.

**P3.5 Policy:** Stem: choose Ready vs high-priority Ready/Suspend. A: swap in Ready/-Suspend only when priority or aging benefit exceeds swap cost; otherwise dispatch resident Ready.

**P3.6 VAX states:** Stem: justify VAX/VMS wait states. A: many waits give precise event/resource queues; paging/global resource waits need no resident/outswapped pair; transitions follow dispatch, wait, event, swap.

**P3.7 Access modes:** Stem: four access modes vs two. A: Kernel, Executive, Supervisor, User give finer least-privilege protection, but add complexity and mode-switch overhead.

**P3.8 Rings:** Stem: ring protection need-to-know problem. A: rings are nested; inner domains can access outer objects, so selective need-to-know access is hard to enforce.

**P3.9 Multi-wait:** Stem: process waiting on multiple events. A: yes, e.g., network, keyboard, timeout; use wait descriptors or multiple queue links and remove stale links after wakeup.

**P3.10 Fixed saves:** Stem: fixed interrupt register-save locations. A: practical only for simple nonnested systems; nested interrupts, multiprogramming, and switches need PCB/kernel-stack context.

**P3.11 Real-time:** Stem: nonpreemptive kernel and real-time. A: it can cause unbounded latency, priority inversion, and missed deadlines; real-time systems need bounded response.

**P3.12 Fork:** Stem: possible output of successful fork(). A: two lines print: 0 in the child and child PID in the parent; order is scheduler-dependent.

### Chapter 3 Fast Recall

- Process = program in execution; PCB makes pause/resume possible.
- Ready lacks CPU; Blocked lacks an event; Suspended is out of main memory.
- Preemption takes CPU from a process before it yields.
- Interrupts are external; traps are instruction-caused; supervisor calls request OS services.
- A mode switch can stay in the same process; a process switch always changes processes.
- Context switching saves old PCB state and restores the selected process state.

### 4 Threads

#### Process vs Thread

**Process**(资源拥有单位): unit of resource ownership and protection; owns address space, process image, open files, I/O resources,

privileges, and IPC links.

**Thread** (调度执行单位) : unit of scheduling/execution; has state, PC, registers, stack, priority, context, and thread-local data. Threads in one process share memory and resources.

**Multithreading** (多线程) : multiple concurrent execution paths inside one process. **PCB** (进程资源) stores process resources; **TCB** (线程上下文) stores thread execution context.

**Advantages** (线程优点): faster create/terminate/switch, cheaper communication, better responsiveness, overlapped I/O and computation, true parallelism on multiprocessors.

**States, Operations, Synchronization**

**Thread states** (线程状态) : Running, Ready, Blocked. Suspend states are usually process-level because all threads share the process address space.

**Operations** (线程操作) : spawn creates a ready thread; block waits for an event; unblock moves to Ready; finish releases context and stack.

**Synchronization** (同步/防共享冲突) : required because shared address space/resources can cause data races, lost updates, and corrupted data structures.

**ULT, KLT, Combined**

**ULT** (用户级线程) : managed by a user-level thread library; kernel sees only the process. Pros: no kernel-mode switch, application scheduling, portable. Cons: blocking syscall may block all process threads; pure ULT cannot exploit multiprocessing.

**KLT** (内核级线程) : managed and scheduled by the kernel. Pros: one blocked thread need not block the process; threads can run on multiple CPUs; kernel code can be multithreaded. Cons: higher switch overhead. **Jacketing** (封套/包裹阻塞调用) : wrap a blocking system call so the thread library can test/yield and avoid blocking the whole process.

**Combined** (混合模型): map many ULTs onto some KLTs/LWPs to mix fast user-level management with kernel parallelism.

**Chapter 4 Review Q&A**

**R4.1 PCB/TCB:** Stem: which PCB elements move to TCB. A: TCB gets thread state, PC, registers, stack pointer, priority, scheduling info, private data; PCB keeps PID, address space, memory, files/I/O, privileges, IPC.

**R4.2 Switch cost:** Stem: why thread mode switch may be cheaper. A: same-process threads keep the same address space and resources; only thread context is saved/restored.

**R4.3 Process traits:** Stem: two independent process characteristics. A: resource ownership and scheduling/execution.

**R4.4 Uses:** Stem: four thread uses. A: foreground/background work, asynchronous processing, faster execution by overlap/parallelism, modular program structure.

**R4.5 Shared resources:** Stem: resources shared by process threads. A: address space, code, global data, heap, files, I/O resources, privileges; each thread has own PC, registers, stack, state.

**R4.6 ULT pros:** Stem: three ULT advantages over KLTs. A: no kernel switch for thread ops, application-specific scheduling, portability.

**R4.7 ULT cons:** Stem: two ULT disadvantages. A: blocking syscall may block the whole process; pure ULT cannot use multiple CPUs in parallel.

**R4.8 Jacketing:** Stem: define jacketing. A: wrapper turns a blocking syscall into nonblocking behavior for user-level thread scheduling.

**Chapter 4 Problem Q&A**

**P4.1 Same-process switch:** Stem: same-process vs different-process thread switch. A: yes; same-process switch avoids address-space/resource switch.

**P4.2 ULT blocking:** Stem: why one blocking ULT blocks all. A: kernel is unaware of ULTs and blocks the whole process on a blocking syscall.

**P4.3 OS/2 sessions:** Stem: remove sessions, bind UI to processes. A: lose grouping of several processes under one UI; assign memory/files/resources to processes, not threads.

**P4.4 1:1 on uniprocessor:** Stem: why faster than single thread. A: when one thread waits for I/O, another ready thread can run; CPU use and responsiveness improve.

**P4.5 Process exit:** Stem: process exits while threads run. A: no thread continues; threads depend on the process address space/resources.

**P4.6 OS/390 ASCB/TCB:** Stem: split global ASCB and local TCB. A: separates address-space management from task execution and reduces global kernel data.

**P4.7 count\_positives:** Stem: function and two-thread risk. A: counts positive list values into a global counter; unsynchronized shared counter creates data-race risk.

**P4.8 Optimized code:** Stem: compiler caches global counter. A: local-register copy

can overwrite another thread update, causing lost update.

**P4.9 Pthreads:** Stem: two threads increment myglobal; output 21. A: unexpected race; read-add-write is not atomic, so increments are lost.

**P4.10 Solaris yield:** Stem: yield to same priority only. A: a higher-priority runnable thread would normally be selected by the scheduler; yield donates only among equal-priority peers.

**P4.11 Solaris ULT/LWP:** Stem: many ULTs per LWP and state diagrams. A: fewer LWPs reduce kernel overhead; ULT state and LWP state differ because user library and kernel schedule different entities.

**P4.12 Linux uninterruptible:** Stem: rationale for Uninterruptible state. A: protect kernel/device consistency while waiting for critical I/O/resource events; wake only when event completes.

**Chapter 4 Fast Recall**

- Process owns resources; thread executes code.
- Threads share process memory/resources but keep private PC, registers, stack, and state.
- ULT is fast and portable but weak on blocking and multiprocessing.
- KLT costs more but supports kernel scheduling, blocking isolation, and CPU parallelism.
- Race condition = unsynchronized shared read/write; lost update is the classic symptom.

**5 Concurrency and Semaphores**

**Concurrency Basics**

**Concurrency** (并发/同一时间段推进) : multiple processes or threads make progress in the same time interval. On one CPU execution is interleaved; on many CPUs it may overlap in parallel.

**Main hazards** (并发风险) : shared global resources, hard resource allocation, and non-deterministic bugs(program hard to reproduce).

**Race condition** (竞态/结果看时序) : final result depends on timing of unsynchronized shared read/write.

**Critical section** (临界区/访问共享资源) : code that accesses a shared resource. **Mutual exclusion** (互斥/一次一个) : at most one process may execute the corresponding critical section for the same resource.

**Interaction and OS Concerns**

|                                     |                                                                                             |
|-------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Unaware</b> (互不知道/竞争)            | competition; issues: mutual exclusion, deadlock, starvation.                                |
| <b>Indirectly aware</b> (间接知道/共享数据) | cooperation by sharing; add data coherence risk.                                            |
| <b>Directly aware</b> (直接知道/通信)     | cooperation by communication; no shared-data mutex, but deadlock/starvation still possible. |

**OS duties** (OS 并发职责) : track processes, allocate/deallocate resources, protect data/resources, and ensure speed independence.

**Mutual Exclusion Requirements**

- Only one process in a critical section for a resource.
- A halted noncritical process must not block others.
- No deadlock or starvation.
- If the critical section is free, entry is not delayed.
- Assume no process speed or processor count.
- A process stays in the critical section only finite time.

**Semaphores**

**Semaphore** (信号量/原子 P V) : integer synchronization variable accessed only by atomic operations: initialize, **semWait**/P/down, **semSignal**/V/up.

**Count meaning** (计数含义) :  $s > 0$  means available units;  $s = 0$  means none free;  $s < 0$  means  $|s|$  blocked waiters.

**Binary semaphore** (二元信号量) : value 0/1, often mutex-like. **General/counting semaphore** (计数信号量) : integer count for multiple resources or ordering.

**Mutex vs binary semaphore** (互斥锁 vs 二元信号量) : a mutex is unlocked by its locker; a binary semaphore may be waited by one process and signaled by another.

**Strong semaphore** (强信号量/FIFO) : FIFO wait queue. **Weak semaphore** (弱信号量/可能饥饿) : no specified release order; starvation is possible.

**Implementation** (实现要点/原子性) : **semWait** and **semSignal** must be atomic; use hardware support, compare-and-swap, short interrupt disable on uniprocessors, or software algorithms.

**Producer/Consumer Patterns**

**Infinite buffer** (无限缓冲/只数产品) :  $n=0$  counts items;  $s=1$  protects the buffer. Pro-

ducer: wait  $s$ , append, signal  $s$ , signal  $n$ . Consumer: wait  $n$ , wait  $s$ , take, signal  $s$ . **Bounded buffer** (有界缓冲/空位+产品) :  $e=size\_of\_buffer$ ,  $n=0$ ,  $s=1$ . Producer: wait  $e$ , wait  $s$ , append, signal  $s$ , signal  $n$ . Consumer: wait  $n$ , wait  $s$ , take, signal  $s$ , signal  $e$ .

Never wait on the resource count while holding the buffer mutex: producer wait  $s$  then  $e$ , or consumer wait  $s$  then  $n$ , can deadlock.

**Chapter 5 Review Q&A**

**R5.1 Design issues:** Stem: four concurrency design issues. A: process communication, resource sharing/competition, synchronization, processor-time allocation.

**R5.2 Contexts:** Stem: three contexts where concurrency arises. A: multiple applications, structured applications, OS structure.

**R5.3 Basic requirement:** Stem: requirement for concurrent execution. A: enforce mutual exclusion for critical resources, with correct synchronization.

**R5.4 Awareness:** Stem: three degrees of process awareness. A: unaware/competition; indirectly aware/cooperation by sharing; directly aware/cooperation by communication.

**R5.5 Compete vs cooperate:** Stem: distinction. A: competing processes only contend for resources; cooperating processes work together by shared data or messages.

**R5.6 Control problems:** Stem: three competing-process problems. A: mutual exclusion, deadlock, starvation.

**R5.7 Mutex requirements:** Stem: list requirements. A: one-at-a-time, noncritical halt harmless, no deadlock/starvation, no needless delay, no speed/CPU assumptions, finite CS time.

**R5.8 Semaphore ops:** Stem: operations. A: initialize, **semWait**/P, **semSignal**/V; no direct arbitrary access.

**R5.9 Binary/general:** Stem: difference. A: binary is 0/1; general/counting is integer-valued for multiple units or synchronization.

**R5.10 Strong/weak:** Stem: difference. A: strong uses FIFO waiter release; weak gives no release-order guarantee.

**R5.11 Monitor:** Stem: monitor. A: high-level synchronization module encapsulating shared data/procedures; only one process active inside, with condition variables for waiting.

**R5.12 Messages:** Stem: blocking vs non-blocking. A: blocking send/receive waits for completion or message; nonblocking returns immediately after queueing or status check.

**R5.13 Readers/writers:** Stem: conditions. A: many readers may read together; writers need exclusive access; policies must avoid reader or writer starvation.

Chapter 5 Problem Q&A

**P5.1 Multi vs multiprocessing:** Stem: two concurrency differences. A: multiprogramming interleaves one CPU and can protect short regions by disabling interrupts; multiprocessing has true overlap, so one-CPU interrupt disable is insufficient.

**P5.3 Race traces:** Stem: print `x` is 10 or `x` is 8. A: interleave the `if` test and `printf` to print 10; split load/decrement/store and load/increment/store to overwrite with stale 8.

**P5.4 Tally bounds:** Stem: two `total()` processes,  $n = 50$ . A: final tally is 2 to 100; with  $k$  processes, upper  $50k$ , lower can still be 2 for  $k \geq 2$ .

**P5.5 Busy wait:** Stem: always less efficient than blocking? A: no; short waits may be cheaper than block/context-switch/restart overhead; long waits should block.

**P5.12 Sem definitions:** Stem: negative-count vs nonnegative-count semaphore definitions. A: equivalent to programs because waiters are represented either by negative count or by the queue; users cannot inspect internals.

**P5.16 General via binary:** Stem: flaw and fix. A: binary `delay` can lose wakeups; use pass-the-baton: if `semSignal` wakes a waiter, it signals `delay` instead of releasing `mutex`; awakened waiter releases `mutex`.

**P5.19 Consumer deadlock:** Stem: move empty-test inside `mutex`. A: consumer can hold  $s$  while waiting on `delay`; producer then waits for  $s$ , so both block.

**P5.20 Faster infinite buffer:** Stem: reduce semaphore overhead. A: let  $n = -1$  mean empty and consumer waiting; producer signals `delay` only when append changes  $n$  from -1 to 0.

**P5.21 Swap order:** Stem: bounded-buffer statement swaps. A: swapping wait  $e/s$  or wait  $n/s$  can deadlock; swapping signal  $s/n$  or signal  $s/e$  preserves correctness but may add waiting.

**P5.22 Trace table:** Stem: fill bounded-buffer chart. A: blank labels: Pa p2, Pb p2, Pb p3, Cb c2, Pc p2, Cb c3, Pa p3, Pc p3, Pa p2, Pa p3; use FIFO queues; final state full = 3, empty = 0, buffer = `XXX`.

Chapter 5 Fast Recall

- Race condition = timing-dependent result on shared data.
- Protect the code that touches shared state, not just the variable name.
- P/down may block; V/up may wake zero or one waiter.
- Semaphore waits before mutex waits in producer/consumer resource

tests.

- Strong semaphore fairness comes from FIFO; weak semaphore may starve.

6 Deadlock and Starvation

Resources and Deadlock Conditions

**Reusable resource** (可重用资源): not depleted by use; released and used again, e.g., CPU, memory, files, devices, semaphores.

**Consumable resource** (可消耗资源): produced and consumed; after use it is gone, e.g., messages, signals, interrupts, buffer information.

**Deadlock** (死锁/互等永久阻塞): processes are permanently blocked because each waits for an event/resource held by another process. **Starvation** (饥饿/一直不被选): a ready/requesting process waits indefinitely because others keep being favored.

**Mutual exclusion** (互斥条件) resource can be held by only one process.

**Hold and wait** (占有并等待) process holds resources while requesting more.

**No preemption** (不可抢占) resource cannot be forcibly taken away.

**Circular wait** (循环等待) closed chain: each process waits for a resource held by the next.

First three conditions make deadlock possible; adding circular wait creates deadlock.

Handling Strategies

**Prevention** (预防/破坏条件): make at least one deadlock condition impossible. Examples: request all resources at once, allow preemption where safe, or impose a global resource order.

**Avoidance** (避免/只给安全状态): grant a request only if the resulting state is safe. Needs each process's maximum claim.

**Detection** (检测/事后发现恢复): grant resources more freely, periodically test the current state, then recover by aborting, rollback, or preemption.

Banker's Algorithm

Need = Claim - Allocation  
For request  $R_i$ : require  $R_i \leq Need_i$  and  $R_i \leq Available$ ; pretend to allocate; grant only if a safe sequence exists.

**Safe state** (安全状态/有完成序列): some process completion order can satisfy all remain-

ing needs. **Unsafe state** (不安全/未必已死锁): no guaranteed safe sequence; deadlock may occur but has not necessarily occurred.

Avoidance vs Detection

**Avoidance** (避免算法) before allocation; uses Claim/Max, Allocation, Available; tests  $Need_i \leq Available$ ; failure means unsafe.

**Detection** (检测算法) after allocation; uses Request  $Q$ , Allocation, Available; tests  $Q_i \leq W$ ; unmarked at end means deadlocked.

**Detection sketch** (检测步骤): set  $W = Available$ ; mark processes with zero allocation; repeatedly find unmarked  $Q_i \leq W$ , mark it, set  $W = W + A_i$ . Remaining unmarked processes are deadlocked.

Chapter 6 Review Q&A

**R6.1 Resource examples:** Stem: reusable vs consumable resources. A: reusable: CPU, memory, files, devices, semaphores; consumable: messages, signals, interrupts, buffer data.

**R6.2 Possible deadlock:** Stem: three conditions for deadlock possibility. A: mutual exclusion, hold and wait, no preemption.

**R6.3 Actual deadlock:** Stem: four deadlock conditions. A: mutual exclusion, hold and wait, no preemption, circular wait.

**R6.4 Prevent hold-wait:** Stem: hold-and-wait prevention. A: require a process to request all needed resources at once, or release held resources before requesting more.

**R6.5 Prevent no-preempt:** Stem: two no-preemption attacks. A: if a request fails, release held resources and retry; or preempt resources from another holder when state can be restored.

**R6.6 Prevent circular wait:** Stem: circular-wait prevention. A: impose a total order on resource types and request only in increasing order.

**R6.7 Three strategies:** Stem: prevention vs avoidance vs detection. A: prevention breaks a condition; avoidance refuses unsafe grants; detection finds existing deadlocks and recovers.

Chapter 6 Problem Q&A

**P6.1 Traffic deadlock:** Stem: show four conditions in Fig. 6.1a. A: each road quadrant is exclusive; cars hold one quadrant while waiting; cars cannot be preempted; cars wait in a cycle.

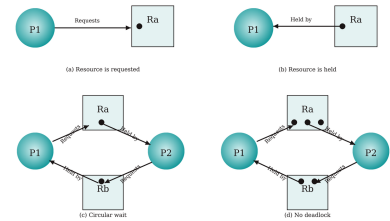
**P6.2 Apply strategies:** Stem: preven-

tion/avoidance/detection for Fig. 6.1. A: traffic lights/order break circular wait; controller allows only safe entry; detection notices blocked cycle and backs/removes one car.

**P6.3 Six paths:** Stem: narrate Fig. 6.3 paths. A: Q-first, P-then-Q, mixed Q-holds-B/P-releases-A, mixed after A release, P-holds-B/Q-waits, or P-first; all avoid a fatal region.

**P6.4 No deadlock:** Stem: justify Fig. 6.3. A: P releases A before requesting B; Q may hold B and wait for A, but P does not hold A while waiting for B, so no circular wait.

**P6.5 Banker state:** Stem: verify, need, safe sequence, P5 request. A: allocated total 9346, so Available 6354. Need rows: P0 7534, P1 2122, P2 3442, P3 2331, P4 4121, P5 3433. Safe order: P1,P2,P0,P3,P4,P5. Granting P5 3233 leaves 3121 and no process can finish; deny.



**P6.6 RAG/order:** Stem: code with P0 A,B,C; P1 D,E,B; P2 C,F,D. A: deadlock cycle P0 waits C held by P2; P2 waits D held by P1; P1 waits B held by P0. Prevent by global order  $A < B < C < D < E < F$ : P0 A,B,C; P1 B,D,E; P2 C,D,F.

**P6.11 Memory banker:** Stem: total 150, P1 70/45, P2 60/40, P3 60/15; add P4. A: initial Available = 50. P4 hold 25 gives Available 25 and safe order P1,P2,P3,P4; grant. P4 hold 35 gives Available 15 and no need fits; deny.

**P6.12 Banker usefulness:** Stem: evaluate banker's algorithm in an OS. A: good theory and predictable-resource systems; often impractical because max claims must be known, resources/processes vary, each request costs a safety test, and concurrency may be reduced.

**P6.15 Min available:** Stem: one resource,  $C = (3, 2, 9, 7)$ ,  $A = (1, 1, 3, 2)$ . A: Need = (2, 1, 6, 5); minimum Available is 3, e.g., P2,P1,P4,P3.

**P6.16 Rankings:** Stem: compare six deadlock methods by concurrency and efficiency. A: concurrency high to low: detect+kill, detect+rollback, banker, ordering, restart-on-wait, reserve-all. Rare-deadlock efficiency: ordering, reserve-all, detect+kill, restart-on-wait, banker, detect+rollback; if deadlocks are frequent, detection recovery becomes less attractive.

Chapter 6 Fast Recall

- Prevention breaks conditions; avoidance checks future safety; detection checks current deadlock.
- Unsafe does not mean deadlocked; it means no guaranteed safe completion order.
- Banker's core loop: find  $Need_i \leq Available$ , finish  $P_i$ , then add back  $Allocation_i$ .
- Detection core loop uses current requests  $Q_i$ , not maximum claims.
- Global resource ordering prevents circular wait but can reduce flexibility.

7 Memory Management

Requirements and Addresses

**Memory management** (内存管理): allocates main memory among active processes and supports efficient multiprogramming.

**Requirements** (五个需求): relocation, protection, sharing, logical organization, and physical organization.

**Logical address** (逻辑地址/程序看到): program-visible address.

**Relative address** (相对地址/偏移): logical offset from a known point. **Physical address** (物理地址/真实主存): real main-memory address.

**Dynamic relocation** (动态重定位): hardware maps relative to physical address, commonly  $physical = base + relative$ ; bounds/limit checks enforce protection.

Partitioning

**Fixed partitioning** (固定分区/内碎片): system-generation split; simple, but internal fragmentation and fixed process count.

**Dynamic partitioning** (动态分区/外碎片): exact-size allocation; no internal fragmentation, but external fragmentation and possible compaction.

**First-fit** (首次适配): first large enough hole; fast and usually good. **Next-fit** (下次适配): resumes from last placement; often uses later holes.

**Best-fit** (最佳适配): smallest large enough hole; often leaves many tiny holes.

**Internal frag.** (内部碎片): waste inside allocation.

**External frag.** (外部碎片): separated holes.

Buddy System and Paging

**Buddy system** (伙伴系统/2的幂): block sizes are powers of two; split on allocation and coalesce free buddies on release. Buddy ad-

dress:  $buddy_k(x) = x \oplus 2^k$ .

**Paging** (分页/页框等大): process logical space is split into fixed-size pages; physical memory is split into same-size frames. Pages need not be contiguous.

**Page table** (页表/页到框): maps page number to frame number. Logical address =  $(page, offset)$ ; physical address =  $(frame, offset)$ .

$$page = \lfloor LA / page\_size \rfloor$$

$$offset = LA \bmod page\_size$$

$$frame = page\_table[page]$$

$$PA = frame \times page\_size + offset$$

Paging has no external fragmentation; only the last page may have internal fragmentation.

## Chapter 7 Review Q&A

**R7.1 Requirements:** Stem: memory-management requirements. A: relocation, protection, sharing, logical organization, physical organization.

**R7.2 Relocation:** Stem: why relocation is desirable. A: processes can be loaded, swapped, or compacted into different physical locations without changing program addresses.

**R7.3 Compile protection:** Stem: why protection cannot be enforced at compile time. A: final physical locations, sharing, and relocation are run-time facts, so run-time checks are needed.

**R7.4 Sharing:** Stem: why share memory regions. A: shared libraries/code, shared data, IPC, common files/buffers, and memory savings.

**R7.5 Unequal fixed partitions:** Stem: advantages. A: better match to process sizes, less internal waste, and support for both small and larger processes.

**R7.6 Fragmentation:** Stem: internal vs external. A: internal is unused space inside an allocation; external is separated free holes outside allocations.

**R7.7 Address types:** Stem: logical/relative/physical. A: logical is program-visible; relative is an offset; physical is the real main-memory location.

**R7.8 Page/frame:** Stem: difference. A: a page is a fixed-size logical-process block; a frame is the same-size physical-memory block.

**R7.9 Page/segment:** Stem: difference. A: pages are fixed-size and system-managed; segments are variable-size logical units such as code, data, or stack.

## Chapter 7 Problem Q&A

**P7.1 Objectives:** Stem: Section 2.3 objec-

tives vs Section 7.1 requirements. A: isolation/protection, allocation/relocation, modularity/logical organization, access control/sharing, and storage hierarchy/physical organization cover the same concerns.

**P7.2 Partition pointer:** Stem:  $2^{16}$ -byte partitions in  $2^{24}$ -byte memory. A:  $2^{24}/2^{16} = 2^8$  partitions, so pointer needs 8 bits.

**P7.3 50 percent rule:** Stem: holes vs segments in dynamic partitioning. A: in equilibrium, random deletion/allocation gives about one hole per two allocated segments, so holes  $\approx n/2$ .

**P7.4 Search length:** Stem: average free-list search. A: best-fit checks all holes unless size-sorted; first-fit and next-fit scan roughly half the list on average.

**P7.5 Worst-fit:** Stem: largest hole placement. A: may avoid tiny leftovers but destroys large holes; search is full list unless kept largest-first.

**P7.6 Diagram:** Stem: X is last 2 MB placement; then one swap-out. A: under low-address splitting, swapped-out max = 1 MB; X split a 7 MB hole; next 3 MB: best-fit exact 3 MB hole, first-fit first 4 MB hole, next-fit 5 MB hole after X, worst-fit 8 MB hole.

**P7.7 Buddy sequence:** Stem: 1 MiB buddy requests 70,35,80; returns A; request 60; returns B,D,C. A: A=128 KiB, B=64 KiB, C=128 KiB, D=64 KiB; after Return B: free A128, free B64, used D64, used C128, free 128, free 512; final all coalesces to 1 MiB free.

**P7.8 Buddy address:** Stem: block address 011011110000. A: size 4 buddy 011011110100; size 16 buddy 011011100000.

**P7.9 Buddy formula:** Stem: general buddy expression. A:  $buddy_k(x) = x \oplus 2^k$ ; equivalently flip bit  $k$ .

**P7.10 Fibonacci buddy:** Stem: use Fibonacci sizes. A: yes; split  $F_{n+2}$  into  $F_{n+1}$  and  $F_n$ . Advantage: less internal fragmentation; cost: harder buddy management.

**P7.11 PC address:** Stem: relative vs absolute PC. A: relative PC is preferable because saved execution state remains relocation-independent; hardware translates on fetch.

**P7.12 Paging bits:** Stem:  $2^{32}$  physical bytes, page  $2^{10}$ ,  $2^{16}$  logical pages. A: logical address 26 bits; frame 1024 bytes; frame field 22 bits; table  $2^{16}$  entries; PTE 23 bits with valid bit.

**P7.13 Translate:** Stem: logical 0001010010111010. A: paging: page 20, offset 186, frame 5 gives 0000010110111010. Segmentation: seg 5, offset 186, base  $4 + 4096 \times 5$ , physical 20670 = 0101000010111110.

**P7.14 Seg table:** Stem: translate  $(seg, offset)$ . A: (0,198)  $\rightarrow$  858; (2,156)  $\rightarrow$  378; (1,530)  $\rightarrow$  fault; (3,444)  $\rightarrow$  1440; (0,222)  $\rightarrow$  882.

**P7.15 Compaction:** Stem: compaction

fraction. A:  $F \geq (1 - f)/(1 + kf)$ ,  $k = t/(2s) - 1$ . For  $f = .2$ ,  $t = 1000$ ,  $s = 50$ ,  $k = 9$ , so  $F \geq 0.8/2.8 \approx 0.286$ .

## Chapter 7 Fast Recall

- Fixed partitions waste inside; dynamic partitions waste between holes.
- Compaction fights external fragmentation but costs CPU and needs relocation.
- Buddy allocation rounds up to a power of two, splits downward, and coalesces upward.
- Paging maps page number to frame number; the offset is copied unchanged.
- Simple paging requires all pages resident; virtual-memory paging can load pages on demand.

## 8 Virtual Memory

### Core Idea and Locality

**Virtual memory** (虚拟内存/只装一部分): lets a process use a large logical address space while only part of it is in real/main memory; the rest is on secondary storage.

**Logical/virtual address** (虚拟地址/CPU生成): generated by the CPU. **Physical/real address** (实地址/主存位置): actual main-memory location. **Address translation** (地址转换): MMU/OS mapping from virtual to physical.

**Resident set** (驻留集/在内存的页): pages of a process currently in memory. **Page fault** (缺页/页不在内存): referenced page is not resident, so the OS must fetch it.

**Locality** (局部性): references cluster in time and space; virtual memory works because the current locality can fit in a modest resident set. **Thrashing** (抖动/忙于换页): page faults dominate execution.

### Paging, TLB, Faults

$$VA = (p, d), \quad PA = (f, d)$$

$$p = \lfloor VA / s \rfloor, \quad d = VA \bmod s$$

$$PA = f \times s + d$$

Offset  $d$  is copied unchanged; the page table maps page  $p$  to frame  $f$ .

**PTE fields** (页表项字段): present/valid bit, modify/dirty bit, reference/use bit, frame number, and protection/control bits.

**TLB** (快表/缓存 PTE): small fast cache of recent PTEs. **TLB miss** (快表未命中) means

the PTE is not cached; **page fault** (缺页中断) means the page is not in memory.

**Fault handling** (缺页处理流程): trap to OS, block process, read page from disk, replace a victim if needed, update PTE/TLB, ready the process, restart the faulting instruction.

### Policies

**Fetch** (取页策略): demand paging loads only on reference; prepaging also loads predicted nearby/future pages.

**Placement** (放置策略): choose a free frame; with paging, placement is usually simple because any frame works.

**Replacement** (置换策略): choose the page least likely to be used soon.

**OPT** (最优/未来最近): evict page used farthest in future; optimal benchmark, not implementable.

**LRU** (最近最少使用): evict least recently used page; good locality match, costly exact implementation.

**FIFO** (先进先出): evict oldest resident page; simple, can evict hot pages and has Belady anomaly.

**Clock** (时钟/二次机会): FIFO plus use bit; use = 1 gets a second chance, use = 0 is victim.

**Enhanced clock** (增强时钟/优先干净页): prefer  $(u, m) = (0, 0)$ , then  $(0, 1)$ , then recently used pages; clean pages are cheaper to evict.

**Resident-set management** (驻留集管理/给几帧): decides how many frames and local/global scope. **Working set** (工作集/近期访问页): pages referenced in recent window  $\Delta$ ; keep it resident to avoid thrashing. **Cleaning** (清除策略/写脏页): demand cleaning writes dirty pages only on eviction; precleaning writes dirty pages early, possibly in batches.

**Page size** (页大小权衡): small pages reduce internal fragmentation but enlarge page tables/TLB pressure; large pages reduce table overhead but may waste memory and weaken locality.

## Chapter 8 Review Q&A

**R8.1 Simple vs VM paging:** Stem: difference. A: simple paging requires all pages resident; virtual-memory paging keeps only needed pages and uses page faults.

**R8.2 Thrashing:** Stem: explain. A: excessive page faults make the system spend most time paging instead of executing.

**R8.3 Locality:** Stem: why crucial. A: refer-

ences cluster, so a small resident set can cover the active working set.

**R8.4 PTE fields:** Stem: typical elements. A: present/valid, modify/dirty, reference/use, frame number, protection/control.

**R8.5 TLB:** Stem: purpose. A: cache recent PTEs to avoid an extra memory access for every translation.

**R8.6 Fetch policies:** Stem: alternatives. A: demand paging loads on fault; prepagging loads likely-needed pages in advance.

**R8.7 Resident/replacement:** Stem: difference. A: resident-set management chooses frame count/scope; replacement chooses the victim page.

**R8.8 FIFO/clock:** Stem: relationship. A: clock is FIFO with a reference bit and second chance.

**R8.9 Page buffering:** Stem: purpose. A: keep replaced pages on free/modified lists for quick reuse and batched dirty writes.

**R8.10 Fixed+global:** Stem: why impossible. A: global replacement changes another process's frame count, violating fixed allocation.

**R8.11 Resident/working set:** Stem: difference. A: resident set is pages actually in memory; working set is pages used recently.

**R8.12 Cleaning:** Stem: demand cleaning vs precleaning. A: write dirty page at eviction vs write dirty pages before they are selected.

## Chapter 8 Problem Q&A

**P8.1 Translate:** Stem: page size 1024, given PTEs; VA 1052, 2221, 5499. A: use  $p = \lfloor a/1024 \rfloor$ ,  $d = a \bmod 1024$ ; results 7196, page fault, 379.

**P8.2 Arrays:** Stem:  $64 \times 64$  arrays, 1 KB pages, 4-page working set. A: column order gives 3 data faults per 4 assignments, total 3072; swap loops to row order gives 3 per 256, total 48.

**P8.3 Table space:** Stem: Fig. 8.3 page table and hashed inverted table. A: normal user page table =  $2^{20} \times 4 = 4$  MB; hashed table =  $64 \times 4 \times 46$  bits = 1472 bytes, about 1.5 KB.

**P8.4 Replacement:** Stem: refs 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 3 frames, count after init. A: FIFO 7 faults/70%; LRU 6/60%; Clock 6/60%; OPT 4/40%.

**P8.5 FIFO anomaly:** Stem: refs A B C D A B E A B C D E. A: FIFO transfers: 3 frames = 9, 4 frames = 10; this is Belady anomaly.

**P8.6 LRU vs FIFO:** Stem: 33-reference trace, 4 frames. A: both LRU and FIFO have 17 faults, 16 hits, hit ratio  $16/33 \approx 48.5\%$ ; equality is trace-specific.

**P8.7 VAX tables:** Stem: user page tables in virtual memory. A: saves real memory; disadvantage is possible page fault while accessing

the page table itself.

**P8.10 Levels:** Stem: 64-bit address, 4 KB pages, 4-byte PTEs. A: offset 12 bits, page number 52 bits, 10 index bits per table page, so  $\lceil 52/10 \rceil = 6$  levels.

**P8.11 EMAT:** Stem: 200 ns memory, 20 ns MMU overhead, 85% TLB hit. A: no TLB = 400 ns; with TLB =  $.85(200) + .15(420) = 250$  ns;  $EMAT = 420 - 200h$ .

**P8.12 Bounds:** Stem: length  $P$ ,  $N$  distinct pages,  $M$  frames. A: lower bound  $N$ ; upper bound  $P$ , or  $N$  if  $M \geq N$ .

**P8.13 Snowplow:** Stem: circular snowplow analogy. A: clock algorithm; pointer clears reference bits as it moves, evicting pages not recently refreshed.

**P8.14 Ref-bit LRU:** Stem: use only reference bit. A: approximate LRU with aging: periodically sample/clear reference bits and evict the lowest age counter.

**P8.18 Address format:** Stem: 32 pages of 2 KB into 1 MB physical memory. A: logical address = 5-bit page + 11-bit offset; table = 32 entries  $\times$  9-bit frame; half memory makes frame field 8 bits, length unchanged.

### Chapter 8 Fast Recall

- Virtual memory = paging plus run-time translation plus page-fault recovery.
- Locality keeps the working set small; poor resident-set sizing causes thrashing.
- TLB miss is a cache miss; page fault is a residency miss.
- Replacement quality: OPT benchmark, LRU strong but costly, FIFO simple but weak, Clock practical.
- Clean victims avoid write-back I/O; dirty victims must be written first.

### 9 Uniprocessor Scheduling

#### Schedulers and Criteria

**Long-term scheduler** (长程/准入作业) : admits jobs to the system and controls the degree of multiprogramming.

**Medium-term scheduler** (中程/交换挂起) : swaps processes in/out of main memory and manages suspended states.

**Short-term scheduler / dispatcher** (短程/派遣 CPU) : chooses the next ready process to run; invoked most often.

**Criteria** (调度评价指标) : response time, turnaround time, throughput, processor utilization, fairness, predictability, deadlines.

$$T_r = finish - arrival, \quad NTT = \frac{T_r}{T_s}$$

**Priority** (优先级/数值级高) : in this text, a smaller priority value usually means higher priority. **Aging** (老化/防饥饿) raises long-waiting jobs to prevent starvation.

#### Decision Modes and Algorithms

**Nonpreemptive** (非抢占) : current process runs until completion or blocking. **Preemptive** (抢占) : OS can interrupt the running process.

**FCFS/FIFO** (先来先服务) - **Full name:** First-Come-First-Served / First-In-First-Out.

**Idea:** nonpreemptive; run the ready process that arrived first or waited longest.

**Pros:** simplest policy; little overhead; fair by arrival order.

**Cons:** convoy effect; short or I/O-bound jobs can wait behind one long CPU-bound job.

**RR** (轮转/时间片) - **Full name:** Round Robin.

**Idea:** preemptive; each ready process gets a fixed time quantum  $q$ , then goes to the queue tail if unfinished.

**Pros:** good response time and fairness for time-sharing systems.

**Cons:** small  $q$  causes many context switches; large  $q$  degenerates toward FCFS.

**SPN/SJF** (最短作业优先) - **Full name:** Shortest Process Next / Shortest Job First.

**Idea:** nonpreemptive; choose the process with the smallest estimated service time.

**Pros:** often minimizes average waiting time when jobs arrive together; favors short jobs.

**Cons:** needs burst-time estimates; long jobs may starve.

**SRT** (最短剩余时间) - **Full name:** Shortest Remaining Time.

**Idea:** preemptive version of SPN; always run the process with the least remaining CPU time.

**Pros:** very good response for short/new short jobs; low average turnaround in many workloads.

**Cons:** needs remaining-time estimates; more preemption overhead; long jobs may starve.

**HRRN** (最高响应比) - **Full name:** Highest Response Ratio Next.

**Idea:** nonpreemptive; choose largest  $R = (w + s)/s = 1 + w/s$ , where  $w$ =waiting time and  $s$ =service time.

**Pros:** short jobs score well, and long-waiting jobs age upward; reduces starvation compared with SPN.

**Cons:** must recompute ratios; still needs service-time estimates.

**Feedback** (反馈/多级反馈队列) - **Full name:** Feedback / Multilevel Feedback Queue.

**Idea:** preemptive; new jobs start in high-priority queues, and jobs that use full quanta are demoted.

**Pros:** no service-time estimate needed; favors short, interactive, and I/O-bound jobs.

**Cons:** CPU-bound jobs can be pushed down and starve unless aging or longer low-level

quanta are used.

### Chapter 9 Review Q&A

**R9.1 Scheduler types:** Stem: three types. A: long-term admits jobs, medium-term swaps processes, short-term dispatches a ready process to CPU.

**R9.2 Interactive OS:** Stem: critical performance requirement. A: response time.

**R9.3 Turnaround/response:** Stem: difference. A: turnaround is submit/arrival to finish; response is submit/request to first visible response.

**R9.4 Priority value:** Stem: low value means what. A: in this text/UNIX style, lower numeric value means higher priority.

**R9.5 Preemption:** Stem: preemptive vs nonpreemptive. A: preemptive can interrupt a running process; nonpreemptive waits for finish or block.

**R9.6 FCFS:** Stem: define. A: ready process waiting longest is selected first.

**R9.7 RR:** Stem: define. A: each ready process gets one time quantum, then returns to the tail if unfinished.

**R9.8 SPN:** Stem: define. A: nonpreemptively select the process with shortest expected service time.

**R9.9 SRT:** Stem: define. A: preemptive SPN; select the process with shortest remaining time.

**R9.10 HRRN:** Stem: define. A: select largest  $(w + s)/s$ , balancing short jobs with aging.

**R9.11 Feedback:** Stem: define. A: multi-level feedback queues demote jobs that consume CPU quanta and favor short/I/O-bound jobs.

### Chapter 9 Problem Q&A

**P9.1 SRT/priority/RR:** Stem: P1-P4 workload, compute schedules and average wait. A: SRT 0-20 : P1, 20-40 : P2, 40-70 : P1, 70-110 : P4, 110-210 : P3, avg wait 25 ms; nonpreemptive priority 0-50 : P1, 50-70 : P2, 70-110 : P4, 110-210 : P3, avg 27.5 ms; RR  $q = 30$ : P1, P2, P1, P3, P4, P3, P4, P3, avg 42.5 ms.

**P9.2 Algorithm comparison:** Stem: A(0,3), B(1,5), C(3,2), D(9,5), E(12,5). A: finish times/mean  $T_r$ /mean NTT: FCFS 3, 8, 10, 15, 20/6.2/1.74; RR  $q = 1$  6, 11, 8, 18, 20/7.6/1.98; RR  $q = 4$  3, 10, 9, 19, 20/7.2/1.88; SPN and SRT 3, 10, 5, 15, 20/5.6/1.32; HRRN 3, 8, 10, 15, 20/6.2/1.74; strict feedback  $q = 1$  7, 18, 6, 19, 20/9.0/2.17, but feedback depends on the stated rule.

**P9.3 SPN proof:** Stem: prove minimum average wait when all jobs arrive together. A: exchange adjacent jobs  $i, j$ ; if  $s_i \leq s_j$ , order

$i, j$  contributes  $2T + s_i$  wait, while  $j, i$  gives  $2T + s_j$ . Repeated swaps yield shortest-first.

**P9.4 Exponential average:** Stem: bursts 6, 4, 6, 4, 13, 13, initial 10. A:  $S_{n+1} = \alpha T_n + (1 - \alpha)S_n$ ; larger  $\alpha$  follows recent bursts faster. For  $\alpha = .5$ : 8, 6, 6, 5, 9, 11, 12; for  $\alpha = .8$ : 6.8, 4.56, 5.712, 4.342, 11.268, 12.654, 12.931.

**P9.5 Bounded estimate:** Stem: roles of  $\alpha, \beta$  in bounded SPN estimate. A:  $\alpha$  controls smoothing vs responsiveness;  $\beta$  scales the estimate before clamping to lower/upper bounds, biasing a job to look shorter or longer.

**P9.6 Feedback variant:** Stem: why A may run 2 or 3 units before B. A: if a new high-priority job immediately preempts a lower-priority running job, A stops after 2; if rescheduling waits for the current quantum to expire, A may run 3.

**P9.7 Minimax RR:** Stem: derive minimax response-ratio scheduling. A: build the order backward; at each step, choose as last the job with the smallest response ratio at the current final completion time, remove it, and repeat.

**P9.8 Minimax proof:** Stem: prove preceding algorithm. A: any schedule's last job has ratio fixed by the final time; choosing the minimum possible last ratio gives the best upper bound, then induction applies to the remaining jobs.

**P9.9 FIFO residence:** Stem: show  $FIFO T_r = T_s/(1 - \rho)$ . A: M/M/1 gives  $W_q = \rho T_s/(1 - \rho)$ ; add service time, so  $T_r = T_s + W_q = T_s/(1 - \rho)$ .

**P9.10 Ideal RR:** Stem: processor-sharing RR with infinitesimal quantum. A: Poisson/exponential processor sharing gives conditional mean response  $R_x = x/(1 - \rho)$ ; RR stretches service by factor  $1/(1 - \rho)$ .

**P9.11 Duplicate PCB pointers:** Stem: same process appears twice in RR queue. A: it gets two quanta per cycle; advantage is weighted RR; equivalent design is a weight field or enlarged effective quantum.

**P9.12 Selfish RR:** Stem: derive selfish RR response time. A: with  $\rho = \lambda s$ ,  $\rho' = \rho(1 - b/a)$ ,  $0 \leq b < a$ ,  $R_x = \frac{s}{1-\rho} + \frac{\pi-s}{1-\rho'}$ .

**P9.13 Dynamic RR quantum:** Stem: divide max response time by ready-process count. A: only a rough heuristic; changing arrivals, swap time, and dispatch overhead can break the response guarantee and make quanta too small.

**P9.14 Feedback favors:** Stem: CPU-bound or I/O-bound. A: I/O-bound/interactive processes, because they block before using full quanta and avoid demotion.

**P9.15 Dekker danger:** Stem: static priority plus busy waiting. A: priority inversion/starvation: a high-priority spinner can keep the low-priority process from running and leaving

the critical section.

**P9.16 Batch turnaround:** Stem: jobs A-E times 15, 9, 3, 6, 12, priorities 6, 3, 7, 9, 4. A: RR  $q = 1$ : A45 B35 C13 D26 E42 avg 32.2; priority order B,E,A,C,D gives A36 B9 C39 D45 E21 avg 30; FCFS gives A15 B24 C27 D33 E45 avg 28.8; SJF order C,D,B,E,A gives A45 B18 C3 D9 E30 avg 21.

### Chapter 9 Fast Recall

- Long-term controls admission; medium-term controls swapping; short-term chooses the next ready process.
- Turnaround =  $finish - arrival$ ; normalized turnaround =  $T_r/T_s$ ; waiting =  $finish - arrival - service$ .
- RR needs a useful quantum: small improves response but raises overhead; large approaches FCFS.
- SPN/SRT improve average wait for short jobs but need burst estimates and may starve long jobs.
- HRRN adds aging through  $1 + w/s$ ; feedback favors short and I/O-bound processes without knowing burst length.

### 10 Multiprocessor, Multicore, Real-Time

#### Multiprocessor Scheduling

**Tightly coupled multiprocessor** (紧耦合多处理器) : processors share common main memory and are controlled by one OS.

**Granularity** (粒度/同步频率) : synchronization frequency. Fine = instruction-level parallelism; medium = cooperating threads in one application; coarse = concurrent processes; very coarse = distributed nodes; independent = unrelated tasks with no explicit synchronization.

**Assignment** (处理器分配) : static assignment keeps a process on one processor, with low overhead but possible imbalance; dynamic assignment uses a common queue, improving balance but hurting cache locality.

**Kernel organization** (内核组织) : master/slave is simple but can bottleneck or fail at the master; peer scheduling scales better but needs stronger synchronization.

**Dispatching** (多处理器派遣) : with multiple processors, simple policies such as FCFS may be adequate because one long job need not block all other work.

| Thread and Core Scheduling                                                                                                                                                        |                                                                                                                                                                                |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Technique (技术)                                                                                                                                                                    | Idea / Pros / Cons (思路/优缺点)                                                                                                                                                    |
| Load sharing (负载共享)                                                                                                                                                               | Idea: global queue. Pros: simple, balanced, keeps CPUs busy. Cons: queue contention, cache-affinity loss, related threads separated.                                           |
| Gang scheduling (组调度/相关线程同跑)                                                                                                                                                      | Idea: co-schedule related threads. Pros: less barrier/lock wait, lower communication delay. Cons: needs CPUs, may waste slots, harder with mixed jobs.                         |
| Dedicated assignment (专用分配)                                                                                                                                                       | Idea: reserve processors per app. Pros: strong affinity, low dispatch cost, predictable progress. Cons: idle when threads block, poor sharing, bad if thread count varies.     |
| Dynamic scheduling (动态调度)                                                                                                                                                         | Idea: vary thread count and processors at run time. Pros: adapts to load, high utilization, can favor urgent work. Cons: complex policy, migration overhead, less predictable. |
| Core placement (核心放置/亲和性)                                                                                                                                                         | Idea: choose cores by cache/memory behavior. Pros: shared-data threads share cache; competitors separated. Cons: needs workload knowledge; may hurt load balance.              |
| <b>Multicore rule</b> (多核经验法则): co-locate data-sharing threads; separate threads that compete for cache or memory bandwidth.                                                      |                                                                                                                                                                                |
| <b>Real-Time Basics</b>                                                                                                                                                           |                                                                                                                                                                                |
| <b>Hard real-time</b> (硬实时/错过不可接受): missing a deadline can cause unacceptable damage or fatal failure.                                                                            |                                                                                                                                                                                |
| <b>Soft real-time</b> (软实时/迟到仍可用): deadlines are desirable but not mandatory; late results may still have value.                                                                  |                                                                                                                                                                                |
| <b>Periodic task</b> (周期任务): occurs repeatedly at regular intervals. <b>Aperiodic task</b> (非周期任务): has no fixed interval but has a start or finish deadline.                     |                                                                                                                                                                                |
| <b>RTOS requirements</b> (实时 OS 要求): determinism, responsiveness, user control, reliability, and fail-soft operation.                                                             |                                                                                                                                                                                |
| Chapter 10 Review Q&A                                                                                                                                                             |                                                                                                                                                                                |
| <b>R10.1 Granularity:</b> Stem: five synchronization granularities. A: fine, medium, coarse, very coarse, and independent parallelism; finer granularity means more frequent syn- |                                                                                                                                                                                |

chronization.

**R10.2 Thread scheduling:** Stem: four thread-scheduling techniques. A: load sharing, gang scheduling, dedicated processor assignment, and dynamic scheduling.

**R10.3 Load sharing versions:** Stem: three load-sharing versions. A: FCFS shared queue; smallest number of threads first; preemptive smallest number of threads first.

**R10.4 Hard vs soft RT:** Stem: hard vs soft real-time tasks. A: hard tasks must meet deadlines; soft tasks prefer deadlines but late completion can still be useful.

**R10.5 Periodic vs aperiodic:** Stem: periodic vs aperiodic tasks. A: periodic tasks recur every period  $T$ ; aperiodic tasks arrive irregularly but still have timing constraints.

**R10.6 RTOS requirements:** Stem: five RTOS requirement areas. A: deterministic behavior, fast response, user control over priorities/resources, high reliability, and graceful fail-soft degradation.

**R10.7 RT algorithms:** Stem: four real-time scheduling classes. A: static table-driven, static priority-driven preemptive, dynamic planning-based, and dynamic best-effort scheduling.

**R10.8 Task data:** Stem: task information useful for real-time scheduling. A: ready time, starting/completion deadline, processing time, resource needs, priority, periodicity, and subtask/dependency structure.

## Chapter 10 Fast Recall

- Static assignment favors affinity; dynamic assignment favors load balance.
- Load sharing is simple but can lose cache affinity and separate cooperating threads.
- Gang scheduling is best when related threads synchronize often.
- Dedicated assignment reduces switching but may waste processors when threads block.
- Multicore scheduling balances load, shared-cache benefit, and resource contention.

## 12 File Management

### Core Model and Organizations

**File system** (文件系统): maps named files and logical records onto secondary-storage blocks. Logical hierarchy: field  $\rightarrow$  record  $\rightarrow$  file  $\rightarrow$  database. Users see files/records; devices transfer blocks.

**File management system** (文件管理系统): OS software for create, delete, open, close,

read, write, protect, share, back up, and recover files.

|                                                                                                                                                                                                                   |                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| <b>File organization</b> (文件组织/逻辑访问): logical record layout/access method; choose by access time, update ease, storage economy, maintenance simplicity, and reliability. top; sequential, indexed, hashed access. |                                    |
| <b>Access method</b> (访问方法层)                                                                                                                                                                                      | record-level operations.           |
| <b>Logical I/O</b> (逻辑 I/O/记录级)                                                                                                                                                                                   |                                    |
| <b>Basic I/O supervisor</b> (基本 I/O 管理)                                                                                                                                                                           | I/O requests, buffers, scheduling. |
| <b>Basic file system / Physical I/O</b> (物理块 I/O)                                                                                                                                                                 | block-level physical I/O.          |
| <b>Device drivers</b> (设备驱动)                                                                                                                                                                                      | lowest; control storage devices.   |

|                                    |                                                                               |
|------------------------------------|-------------------------------------------------------------------------------|
| <b>File</b> (堆文件/追加无序)             | unordered arrival order; simple append, slow search.                          |
| <b>Sequential</b> (顺序文件/按键排序)      | records sorted by key; strong for batch/full scans.                           |
| <b>Indexed sequential</b> (索引顺序文件) | sorted file plus index and overflow; faster lookup and sequential processing. |
| <b>Indexed</b> (索引文件/多索引)          | one or more indexes; flexible random access, index overhead.                  |
| <b>Direct-hashed</b> (直接/散列文件)     | fastest exact-key lookup, weaker for ordered scans.                           |

### Directories, B-Trees, Blocking

**Directory** (目录/文件元数据表): a file containing file metadata: name/type/org, volume/start/size, owner/access rights, timestamps, open/lock state. Ops: search, create, delete, list, update.

**Pathname** (路径名/绝对相对): absolute starts at root; relative is interpreted from the working/current directory. Tree directories avoid name conflicts by unique pathnames. **Access rights** (访问权限): none, knowledge, execution, reading, appending, updating, changing protection, deletion.

**B-tree** (B 树/平衡多路索引): balanced multiway search tree; all leaves same depth, broad and shallow. Minimum degree  $d$ : max keys  $2d - 1$ , max children  $2d$ , nonroot min keys  $d - 1$ . On full-node insert, split and promote median.

|                                   |                                                                                                                                                                                                                                                                                               |
|-----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Fixed blocking</b> (固定分块)      | Idea: fixed records, integral records/block. Pros: simplest, easy addressing. Cons: internal fragmentation; poor for variable records. Idea: variable records may cross blocks. Pros: best space use; record may exceed block. Cons: complex I/O/update; one record may need two block reads. |
| <b>Variable spanned</b> (可变跨块)    | Idea: variable records cannot cross blocks. Pros: simpler; one record stays in one block. Cons: leftover waste; record $\leq$ block.                                                                                                                                                          |
| <b>Variable unspanned</b> (可变不跨块) |                                                                                                                                                                                                                                                                                               |

### Allocation and Free Space

|                          |                                                                                                                                                                         |
|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Contiguous</b> (连续分配) | Idea: start block + length. Pros: fastest sequential/direct access; tiny metadata. Cons: external fragmentation; hard growth; size often needed early.                  |
| <b>Chained</b> (链接分配)    | Idea: each block points to next. Pros: no external fragmentation; easy growth. Cons: slow random access; pointer overhead; poor locality.                               |
| <b>Indexed</b> (索引分配)    | Idea: index block lists data blocks. Pros: good random/sequential access; flexible growth. Cons: index overhead; small-file waste; large files need multilevel indexes. |

**Preallocation** (预分配): reserve max size early. Pros: simple, may be contiguous/fast. Cons: wastes space; max size hard to know.

**Dynamic allocation** (动态分配): allocate as file grows. Pros: space efficient, flexible. Cons: more metadata; may fragment.

**First fit** (首次适配): first large-enough free portion. Pros: fast. Cons: small holes near front. **Best fit** (最佳适配): smallest large-enough portion. Pros: least immediate leftover. Cons: slow search, tiny fragments. **Nearest fit** (邻近适配): closest to prior file allocation. Pros: locality. Cons: not space-optimal; needs position state.

**Free-space management** (空闲空间管理): bit table/bitmap, chained free portions, indexing, free block list. Bitmap bytes =  $\text{disk bytes} / (8 \times \text{block size})$ .

**Do not confuse** (易混点): indexed file = logical file organization; indexed allocation = physical block allocation.

### Chapter 12 Review Q&A

**R12.1 Field/record:** Stem: difference. A: a field is one data item; a record is related fields treated as one unit.

**R12.2 File/database:** Stem: difference. A: a file is similar records; a database is related data, often several files with explicit relationships.

**R12.3 FMS:** Stem: define file management system. A: OS software that provides file creation, deletion, access, modification, protection, sharing, backup, and recovery.

**R12.4 Organization criteria:** Stem: choosing file organization. A: short access time, ease of update, storage economy, simple maintenance, and reliability.

**R12.5 Five organizations:** Stem: list and define. A: pile = arrival/full scan; sequential = key-sorted/batch; indexed sequential = index+overflow/faster lookup; indexed = multiple indexes/fields; direct/hashed = hash key to location.

**R12.6 Indexed sequential search:** Stem: why faster than sequential. A: the index jumps near the target, so only a small local sequential search remains.

**R12.7 Directory ops:** Stem: typical operations. A: search, create file, delete file, list directory, and update directory.

**R12.8 Path/working dir:** Stem: relationship. A: a relative pathname is resolved from the working directory; an absolute pathname starts at root.

**R12.9 Access rights:** Stem: typical rights. A: none, knowledge, execution, reading, appending, updating, changing protection, deletion.

**R12.10 Blocking:** Stem: three methods. A: fixed; variable-length spanned; variable-length unspanned.

**R12.11 Allocation:** Stem: three methods. A: contiguous, chained, and indexed file allocation.

## Chapter 12 Problem Q&A

**P12.1 Blocking factor:** Stem:  $B, R, P$ =block/record/pointer sizes;  $F$ =blocking factor. A: fixed/unspanned  $F \approx \lfloor B/R \rfloor$ ; spanned  $F \approx (B - P)/R$ .

**P12.2 Growing portions:** Stem: portions 1, 2, 4, ...,  $n$  records, blocking factor  $F$ . A: let  $m = \lceil n/F \rceil$  blocks; FAT entries  $\leq \lceil \log_2(m + 1) \rceil$ ; just after a new portion, unused space is at most that portion minus one block/record unit, less than half allocated.

**P12.3 Choose organization:** Stem: choose org for speed/storage/update: (a) infrequent update + frequent random access; (b) frequent update + frequent whole-file access; (c) frequent update + frequent random access. A: (a) direct/hashed, or indexed for multiple

keys; (b) indexed sequential; (c) indexed, or hashed for one exact key.

**P12.4 Insert 97:** Stem: B-tree insert. A: insert into full right leaf, split [73, 85, 88, 90, 96, 97] at 88; root also full, split at 51; new root [51], right child includes [61, 71, 88].

**P12.5 Top-down split:** Stem: split every full node while descending. A: advantage: no backtracking and simpler one-pass insertion; disadvantages: unnecessary splits, extra writes, lower occupancy, possible earlier height growth.

**P12.6 B-tree height:** Stem: upper bound on height. A: worst case has minimum occupancy, so  $n \geq 2d^h - 1$  and  $h \leq \log_d((n+1)/2)$ , with root-to-leaf edge height.

**P12.7 Last-block waste:** Stem: 16K blocks;  $S = 41,600; 640,000; 4,064,000$ . A:  $A = \lceil S/16K \rceil 16K$ ,  $W\% = (A - S)/A$ ; 15.36%, 2.34%, 0.383%.

**P12.8 Directory benefits:** Stem: advantages. A: symbolic names, name-to-location mapping, hierarchy, fewer name conflicts, access control, easier search/list/update.

**P12.9 Directory file type:** Stem: special file vs ordinary file. A: special protects format/integrity but is less flexible; ordinary gives uniform tools but risks corruption unless strongly protected.

**P12.10 Limited depth:** Stem: effect of shallow tree. A: users lose deep organization and long paths; OS gains simpler pathname parsing, bounded buffers/tables, and less recursive traversal.

**P12.11 Lost free pointer:** Stem: hierarchical FS keeps free disk space in a free-space list: (a) if pointer to free space is lost, can system reconstruct the list? (b) scheme so pointer is never lost from one memory failure. A: scan directories/FAT/inodes, mark allocated blocks, rebuild unmarked free blocks; store redundant logged/checksummed pointer copies in stable storage.

**P12.12 UNIX V max file:** Stem: In UNIX System V, block length is 1 KB and each block holds 256 block addresses; using the inode scheme, find the maximum file size. A: with 10 direct plus single/double/triple indirect,  $\text{max} = (10 + 256 + 256^2 + 256^3) \times 1024 = 17,247,250,432$  bytes, about 16.06 GiB.

**P12.13 8K inode:** Stem: UNIX inode: 12 direct pointers plus single/double/triple indirect; block=sector=8K; 32-bit disk pointer = 8-bit disk ID + 24-bit block ID. Find (a) max file size, (b) max partition size, (c) disk accesses for byte 13,423,956 when inode is already in memory. A: pointers/block = 2048;  $\text{max file} = (12 + 2048 + 2048^2 + 2048^3) 8K = 70,403,120,791,552$  bytes. Partition limit is 128 GiB per 24-bit block field per disk, or 32 TiB using all 8 disk bits. Byte 13,423,956 is

in the single-indirect range, so read indirect block + data block = 2 disk accesses.